



Important Contest Instructions !!

Please read the following instructions carefully. They contain important information on how to run your programs and how to prepare them for submission. If you have any questions regarding these instructions, please ask it in our Discord server (your teacher has the link).

Java

Your solution file name must be **probXX.java** and your class name must be **probXX**, where **prob** is all *lower case* and **XX** corresponds to the two digit problem number. (ex. `public class prob01, prob01.java`). Remember to copy the entire Java source file contents into the submission box.

C / C++

We are accepting **C/C++ source code**. Any solutions in C or C++ must be submitted as source files and not as a binary executable. Your solutions can rely on glibc/libm and the standard headers.

Download our "C/C++ Development Environment Guide" from <https://codewarscentral.org/downloads>

Program Input (all languages)

Most solutions will require input. You have two options:

File Input

Your program may read input from a file named **input.txt** for all problems. The file will be in the same directory as your program.

Download our "Guide to data file I/O" from <https://codewarscentral.org/downloads>

Keyboard Input

Your program may read input from standard in (the keyboard). Do **not** include any prompts in your output. There are two options to provide input to your program via standard in (the keyboard).

- (1) The preferred option is to redirect the contents of a file to standard in of your program.

```
java prob01 < input.txt
java -jar js.jar prob01.js < input.txt
python prob01.py < input.txt
prob01.exe < input.txt
```

- (2) Otherwise you may type everything manually, or copy/paste from the packet or data set files.

Tip: Type Ctrl-Z <return> to signal the end of keyboard input.

Program output (all languages)

Your program must send the output to the screen (standard out, the default for any print statement). Do **not** include any prompts for input in the output! The only output for your program should be the expected output for the given problem.

Testing your program

Test your program like a judge will with the Windows Batch script (`checkProb.bat`) in the student ZIP file. It tests your program with all student data sets for a given problem. It will redirect the data set file contents to standard in and create the **input.txt** file.

Online Code Submission

Solutions will be submitted through the contest site using a text box (with a large character limit, don't worry!) instead of a file upload. You will need to copy and paste the solution from your local source code file into the text box. Once you select the problem number and language from the respective drop-downs, you can submit the solution for judging. For Java, the main class must still be named **probXX**.

This year Hogwarts will play host to a legendary event: the Triwizard Tournament. During which time a single student gets to represent his or her school in a series of magical contests.

Eternal glory awaits the student who wins the tournament. Please join me in welcoming our guests!



Input

You will not receive any input for this (and only this) problem

Output

Print a welcome message to the screen as follows:

```
We bid welcome to the ladies of Beauxbaton and our friends from the north, the sons of Durmstrang.
```

Discussion

Simply print to the screen the sentence given above as output. There are no datasets for this problem (as it is meant to ensure that your coding environment and setup works when submitting code for judging).

Help Spock welcome you, the new captain of the Enterprise, to the bridge with a reminder of your duty -- by telling him your name.



Input

You will receive exactly one word on a single line.

```
Jim
```

Output

Print the following sentence to the screen, substituting the word read in from your dataset in place of the placeholder NAME: *NAME, the needs of the many outweigh the needs of the few, or the one; live long and prosper.*

```
Jim, the needs of the many outweigh the needs of the few, or the one; live long and prosper.
```

Discussion

You do not have to worry about spaces, hyphenated names, or words that are too long. You are guaranteed that the words supplied in the datasets will fit in the output, and conform to the guidelines given.

Reminder: have you run your solution against **all** of the student data sets?

Additional Examples

Input 1	Output 1
Kate	Kate, the needs of the many outweigh the needs of the few, or the one; live long and prosper.
Input 2	Output 2
Sisko	Sisko, the needs of the many outweigh the needs of the few, or the one; live long and prosper.

Nothing is better than Friday night because you get to eat some pizza. One thing about pizza is they are not all the same size. Pizzas aren't just circles they have a thickness too.

It turns out that the mathematical volume of a pizza is pizza. How does that work you say? Well if z = radius of the pizza and a = the height then $\pi * \text{radius} * \text{radius} * \text{height} = \text{Pi} * z * z * a = \text{Pizza}$.

Mathematically, the volume V is calculated by multiplying pi ' π ' the height h and the radius times the radius z .

$$V = \pi * z * z * h \quad (\pi = 3.1415926536)$$

Write a program to calculate the volume of a pizza given its height and radius.

Input

The input consists of two decimal numbers:
the height of the pizza in inches and the radius of the pizza in inches.

Use the height and radius to calculate the volume of the pizza in cubic inches.

```
16.50 24.00
```

Output

Print volume of the pizza, including two decimals, then a space and then "cubic inches"

```
29857.70 cubic inches
```

Discussion

Round to nearest hundredth.

Additional Examples

Input	Output
1.00 7.50	176.71 cubic inches

Superheroes Don't Always Wear Capes



Ace Ventura, Pet Detective, is on the hunt for a rare albino pigeon. Help Ace find the pigeon with your drone's new mapping app.



Input

You will receive a map grid made up of lines of ASCII characters. The map will be between 5-40 lines tall, and 5-80 characters wide. Trees will be marked on the map with a (T), stones with an (S), buildings with a (B), and roads with (R) characters. The pigeon, if the drone can spot it, will be marked with a (P). All other empty spaces on the map will be marked with a period (.)

```
....R.....
....R.T.....BB...
....R.T.....BB...
.S..R.....
....R.T.....BB...
....R.T.....BB.P.
....R.....
....RRRRRRRRRRRRRR
```

Output

If the drone marked the map with a (P) character, quickly tell Ace where the pigeon is by texting him the coordinates from the map. The upper left of the map will be (X=0,Y=0). The lower right of the map will be the maximum values for X and Y. If the drone couldn't spot the pigeon on the map, text Ace to try another map (see below).

```
Ace, move fast, pigeon is at (18,5)
```

Output the full sentence exactly as shown with only the map coordinates changing if your output found a pigeon. Else, output the sentence:

```
No pigeon, try another map, Ace
```

Additional Examples

Reminder: have you run your solution against **all** of the student data sets?

Input 1	Output 1
RRRRR .T..T ..S.. RRRRR .B..B	No pigeon, try another map, Ace



Joe Gardner teaches a music class at a school and needs your help. He is in need of some noise canceling headphones to prevent headaches during some of his classes. Joe doesn't want to buy the headphones because he wants to custom make them into some of his clothes so that his students won't know. Could you help Joe by inverting (flip) the sound wave that is coming into the headset?

Input

You will receive the number of lines you will receive followed by #'s in a wave pattern. Note, there are no flat parts to the waves. Max Size of the wave is 80x80.

```
7
#
# #      #
# #      # # #
#  #      # # #
      # #      #
      # #
      #
```

Output

The output should be the inverted (flipped) wave.

```
      #
      # #
    #  #      #
#  #      # # #
#  #      # # #
  # #      #
  #
```

Discussion

Noise Canceling works by using microphones to pick up low-frequency noise and neutralize it before it reaches the ear. The headset generates a sound that's phase-inverted (x-axis flip) by 180 degrees to the unwanted noise, resulting in the two sounds cancelling each other out.

The Flip always occurs on the X axis.

Additional Examples

Input

```
8
#           #
#  #       # #           #
# # #     #  #   #  # #
#  #     #   #  # # #  #
#           #  #  #  #  #
#  #     #  #  #  #  #
#  #     #           #
#  #     #           #
#  #     #           #
#           #
```

Output

```
          #
        # #
      #  #
    #    #   #   #
  #  #   #  #  #  #
#  #   #  #  #  #  #
# # #   #  #   #  #
#  #   #  #   #
#           #
#           #
```


Shang needs your help. His bosses just imposed another rule on who can enlist into the military.

He cannot have anyone under his command who has the same birthday (day and month only)! Can you please check through the list of birthdays and tell him how many duplicate sets there are?

Note: Multiple people with same birthday are counted only once while counting duplicate sets.

Input

The input will be in the format mm/dd/yyyy

```
7
01/09/2000
01/10/2000
01/09/2000
01/09/2000
01/09/2000
01/11/2003
01/11/2002
```



Output

Output should have the first line indicating the amount of doubles. The next line should list the dates, without the year, that are duplicates in the order they appeared in the list.

```
2
duplicates: 01/09 01/11
```

Discussion

The answer to the example is 2, because 01/09 is shared by 4 people and 01/11 is shared by 2, so there are 2 dates that have duplicates. The second line shows the shared dates (without the year): "duplicates: 01/09 01/11". Make sure to put spaces between the dates.

If there are no duplicates, please print a 0 and "duplicates: None":

```
0
duplicates: None
```

Additional Examples

Input 1	Output 1
10 01/01/2000 02/10/2000 01/23/2000 05/09/2000 01/04/2000 09/11/2003 10/11/2002	0 duplicates: None

Eve is at it again and may have discovered a new way to intercept and read encrypted WiFi network traffic at her local coffee shop.

She has determined the following mapping by performing a few tests:

```
PLAINTEXT  ABCDEFGHIJKLMNOPQRSTUVWXYZ
CIPHERTEXT FGHIJKLMNOPQRSTUVWXYZABCDE
```

The following are examples of ciphertext she has painstakingly decoded so far:

```
PLAINTEXT  HELLO WORLD
CIPHERTEXT MJQQT BTWQI
```

```
PLAINTEXT  CODE WARS RULES
CIPHERTEXT HTIJ BFWX WZQJX
```

```
PLAINTEXT  SUPERCALIFRAGILISTICEXPIALIDOCIOUS
CIPHERTEXT XZUJWHFQNKWFLNQXNYNHJCUNFQNITHNTZX
```



In order to read data in realtime, Eve needs an algorithm. Can you provide an algorithm to decode any ciphertext into its plaintext equivalent?

Input

Data is provided in two lines. The first line contains the length of the ciphertext. The second line contains the full snippet of ciphertext to be decrypted.

```
11
MJQQT BTWQI
```

Output

Print the decoded plaintext.

```
HELLO WORLD
```

Discussion

- Whitespace is whitespace (in ciphertext or plaintext, it maps to the same value.)
- Ciphertext will only be the upper-case alphabet characters listed in the table above.
- All text will be written using upper-case (both cipher and plaintext).

Having conquered the world of extreme underwater basket weaving, you are now moving on to Xtreme Surface Skiing! As the most extremely experienced X-games competitor in your new league, they are seeking your expertise to decide what materials to use for the first round of competitions (before the shark jumping round).

Input

You will receive on a single line a SKI MATERIAL ("X" coordinate in table below, A.K.A. the top row), followed by a skiing SURFACE ("Y" coordinate in table below, A.K.A. the first column). Use the material-to-surface lookup table in the discussion section to determine the coefficient of friction for the given ski material on the given surface. Use that coefficient of friction to find the minimum slope angle (using the inverse tangent (known as arctan)) the league would need to use for the competition.

WOOD RUBBER



Output

Output the coefficient of friction you found in the lookup table followed by a space and minimum slope the league would need to use for Xtreme skiing on those materials, rounded to the nearest whole tenth. You are guaranteed that all materials given as inputs will have corresponding outputs that are real, non-negative numbers.

0.80 38.7

Discussion

Coefficients of Friction Lookup Table

There are 5 skiing surfaces and 3 ski materials.

SURFACE	SKI MATERIAL		
	RUBBER	WOOD	STEEL
CONCRETE	0.90	0.62	0.57
WOOD	0.80	0.42	0.30
STEEL	0.70	0.30	0.74
RUBBER	1.15	0.80	0.70
ICE	0.15	0.05	0.03

The arctan of a given coefficient of friction will output, in degrees, the corresponding angle which would equal/overcome it and allow sliding to occur. Use your programming language's math library's arctan function (all allowed languages for CodeWars have one). **Make sure you are calculating in degrees, not radians!**

In the example given above, the SKI MATERIAL is WOOD, and the SKIING SURFACE is RUBBER. Looking up a SKI MATERIAL of WOOD and a SURFACE of RUBBER in our table, we find the coefficient of friction to be 0.80. Arctan(0.80) is 38.65980825 degrees. Rounding that up to the nearest tenth we get: 38.7.

Reminder: have you run your solution against **all** of the student data sets?

Additional Examples

Input 1	Output 1	Input 2	Output 2
RUBBER RUBBER	1.15 49.0	STEEL ICE	0.03 1.7

King Candy has ordered Sour Bill, Wynnchel and Duncan to build candy walls to slow down Ralph -- who is on the way to the castle to save Vanellope.

Ralph is angry. The angrier Ralph is, the harder he hits!

Show Vanellope what she will see outside the window in her cell as Ralph rushes to save her.



Input

You will receive three integers, separated by spaces. The first integer tells you how tall (max height of 40) the candy wall is (use the hash character (#) to show walls). The second integer tells you how high Ralph punches. The third integer tells you how mad Ralph is on a scale of 1 to 99, where 1 is annoyed someone is talking on their phone, and 99 is strong enough to destroy the arcade.

```
6 3 11
```

Output

Show the aftermath of Ralph punching the wall. Every 10 levels of angry will cause the destruction of the wall to be one additional block more violent.

At anger level 1 the wall will fall down right next to where it was standing. At level 21 the wall will get knocked 2 spaces away from where the wall was standing before it falls over. (Use a period to show the spaces between the original wall, and the fallen wall) The part of the wall below where Ralph punches will remain unchanged.

If Ralph is so angry he misses, the second integer will be zero (to show a miss) -- in that case, simply show the wall unharmed. Ralph can *also* miss by punching **higher** than the wall is tall. Ralph is too tall to punch block number 1.

```
#
# .####
```

Discussion

Taking the example from above, let's break it down into before and after pictures. The wall before it is punched should be 6 "blocks" tall. Then Ralph hits it at block-height 3, with an anger-force of 11. From 0-9 the wall should just topple over to the right where it is hit. At 10, 20, 30 ... 90, we need to add an additional period between the remainder of the wall Ralph didn't punch, and the start of the toppled over wall on the ground. Since Ralph hits with a force of 11, that means we crossed the 10-threshold, so we need to add one period after the wall before we start scattering blocks on the ground.

Before	After
#	
#	
#	
#	
#	#
#	#.####

Reminder: have you run your solution against **all** of the student data sets?

Additional Examples

Input 1	Output 1	Input 2	Output 2
4 2 9	####	5 0 99	# # # # #
Input 3	Output 3	Input 4	Output 4
3 2 39	#...##	7 8 20	# # # # # # #

The minions need to help Gru build a larger jet because they can't all fit in his current one. Kevin, Stewart and Bob are drafting plans for a 4-piece jet which has a spherical cockpit, a cylindrical body, and two pyramid-shaped storage pods (one on either side). Help Gru run the calculations for how many minions he can pack inside the proposed space for a design, and either approve or reject the plan.

Input

You will receive exactly 4 lines of input. The first line will be the total number of Minions Gru's new jet needs to hold. The second line will be the proposed radius of the cockpit. The third line will be the proposed radius and height of the body (in that order, separated by spaces). The fourth line will be the proposed length, width and height (in that order, separated by spaces) of the twin storage pods. All measurements will be given in meters as floating point decimals.



```
872
15.90
22.10 17.34
10.00 10.00 9.82
```

Output

You are looking for the AVAILABLE space in the proposed design. Note: Minions can squeeze themselves down into relatively small spaces as needed, and they are squishy, they can *fill* any available space as well. The minimum space one minion needs is 1.20 m^3 . Run the calculations for the volume of the parts of proposed jet plan, with the following caveats:

- Gru *also* takes up space. 2.20 m^3 of space will be needed in the cockpit for Gru
- The controls in the cockpit *also* take up space. 4.10 m^3 of space is needed for the controls
- In the body of the jet some emergency equipment, and additional flight controls *also* take up 12.10 m^3 of space.
- The storage pods are completely empty, and can be crammed to the rafters with Minions

Output the remaining **available** space in each area of the proposed jet plan (rounded to 2 decimals), the total space the Minions need, and then either a PLAN ACCEPTED or PLAN REJECTED after the summary, if there is (or isn't) enough space for all of the Minions inside the proposed plan.

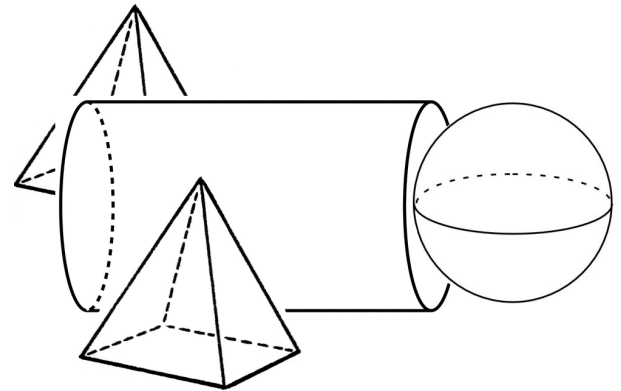
```
Cockpit 16831.29
Body 26594.14
Pods 654.67
Minions Need 1046.40
PLAN ACCEPTED
```

Discussion

Use 3.1415926536 to represent pi if your programming language doesn't have a built-in representation of pi. That level of precision will be okay for this problem. Also note, that due to rounding differences between the various languages, we will accept +/-0.01 for the numbers given in your output.

Formulas:

- Volume of a sphere: $V = \frac{4}{3}\pi r^3$
- Volume of a cylinder: $V = h\pi r^2$
- Volume of a pyramid: $V = \frac{1}{3}lwh$



Reminder: have you run your solution against **all** of the student data sets?

Additional Examples

Input 1	Output 1	Input 2	Output 2
1032 3.70 3.33 11.01 4.00 4.00 5.14	Cockpit 205.87 Body 371.45 Pods 54.83 Minions Need 1238.40 PLAN REJECTED	455 3.00 3.00 11.00 6.00 7.00 5.01	Cockpit 106.80 Body 298.92 Pods 140.28 Minions Need 546.00 PLAN ACCEPTED

Dracula is shocked that the villagers have formulated a vampire-block cream which provides protection against vampire bites. The cream is sold in 20,50,75,100 and 150 VPF (vampire protection factor) bottles. A VPF of 1 provides 10 minutes of vampire bite protection. It is 5 PM (17:00) and Dracula is hungry. Help him figure out if he needs to order *delivery* for the night or if a villager has been lax in applying their vampire-block cream.

Input

You will receive up to 25 lines in a file. Each line will give the following information: Villager-## HH:MM NN Where ## is a 2-letter code for the villager (Dracula can't be bothered to learn the names of his snacks), HH:MM is the time (in 24-hour format) the villager applied their vampire-block, and NN is the VPF they used. END on a line by itself signifies the end of input.

```
Villager-AA 08:00 50
Villager-AB 14:30 20
Villager-AC 00:19 100
Villager-AD 10:01 150
END
```

Output

Figure out, based on the time the cream was applied, and how long the VPF will last, if Dracula can snack on a villager or not. (Protection means their VPF will last until a time **greater than or equal to 17:00**). For each villager found to not have enough vampire-block on, list them as tasty in the output as a single line grouped by their 2-letter codes in the order of their appearance in the input list (if there is only one villager to list, don't worry about changing the "Villagers" to "Villager").

```
Villagers (AA, AC) look tasty
```

If no villagers were lazy today, print:

```
Blah, blah, blah, time to order delivery
```

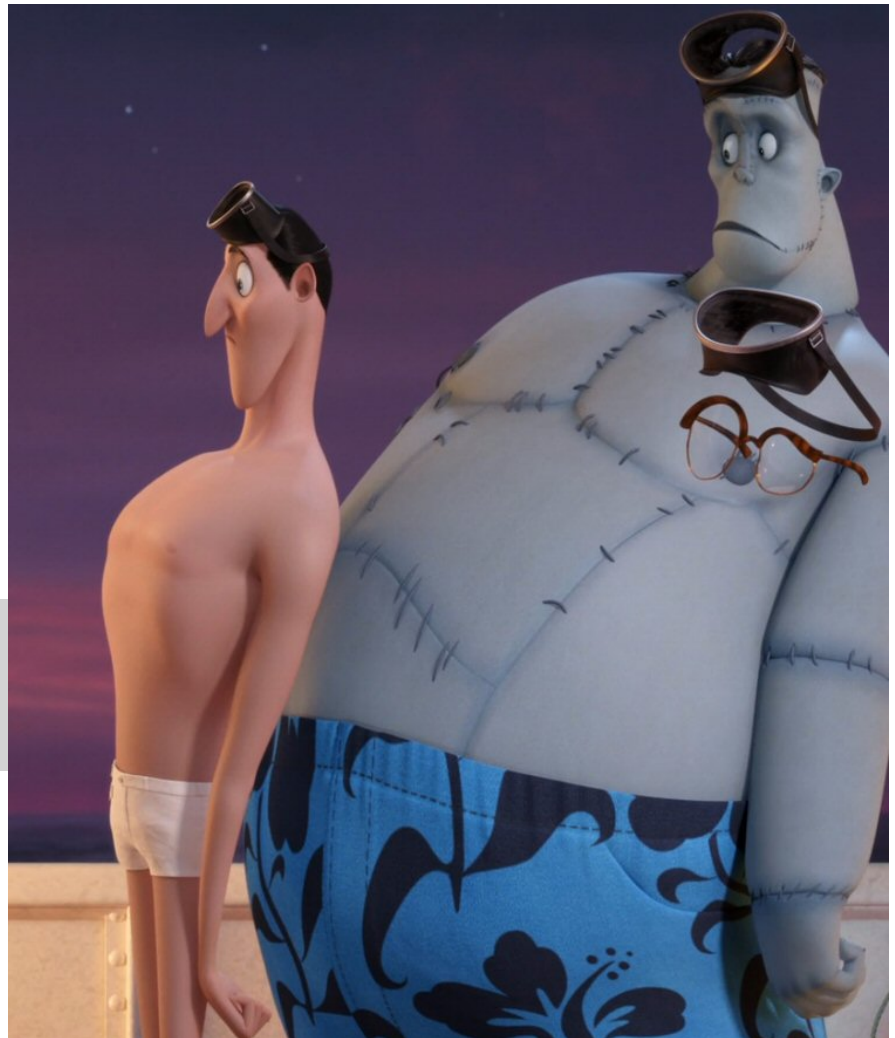
Discussion

In the Example above, AA starts at 8:01 with VPF 50 (500 minutes), expiring at 16:21, so is tasty at 17:00. AB starts at 14:30 with VPF 20 (200 min), expiring at 17:50, so is protected at 17:00. AC starts at 00:19 with 1000 min, expiring at 16:59, so is tasty at 17:00. AD starts at 10:01 with 1500 min, expiring much later than 17:00, so is protected.

Reminder: have you run your solution against **all** of the student data sets?

Additional Examples

Input	Output
Villager-QZ 09:01 50 Villager-HJ 07:45 150 Villager-MW 11:01 175 END	Blah, blah, blah, time to order delivery



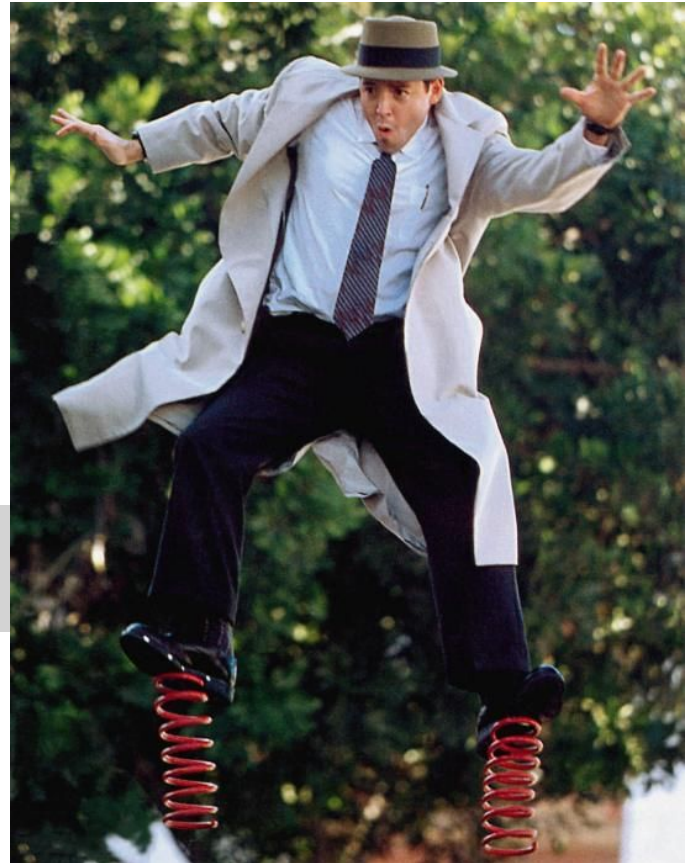
Inspector Gadget just installed new springs in his feet and they do not feel like they should, making it where he cannot stop!

However the Inspector cannot do the math while stopping himself from crashing into things. Using Hooke's Law, $F = -kx$, please help him find out the F, k, or x value as he requests it.

Input

You will receive 3 lines of input. Each line will start with the variable (capital letters only) then the value of that variable. The value will always be a decimal number between 0 and 10,000 (or a question mark).

```
F 10.00
K 3.80
X ?
```



Output

You should output the variable, a space, then the value you found for it. Round the output to the nearest hundredths place.

```
X -2.63
```

Discussion

The missing data that you need to find will be shown as a '?' instead of a number. You will always receive 3 lines and the variables will always be in the same order, F, K, then X. There will always be 1 "?" and the "?" will not always be after the same variable.

Additional Examples

Input 1	Output 1	Input 2	Output 2	Input 3	Output 3
F 9.00 K -3.80 X ?	X 2.37	F ? K 3.00 X 2.51	F -7.53	F 8.88 K ? X 12.70	K -0.70

The Allies are trying to get a message **25 meters** straight up a cliff to a waiting team in the cyberpunk virtual wars of 7702 (spoiler alert, it isn't going well). They are trying to build a contraption which will get a messenger up the cliff to deliver the message in person (mail is expensive in the year 7702, so . . . contraptions!)

Input

You will receive, on a single line separated by spaces, the name of the contraption the Allies want to build, the speed to which it can launch something into flight and the distance per time unit that the contraption can launch something. You may receive the following units: MILES, KILOMETERS, YARDS, FEET, METERS, INCHES, CENTIMETERS, HOUR, MINUTE, SECOND.

```
GIANT-SLINGSHOT 1980.00 FEET PER MINUTE
```



Output

Calculate (using the formula given below) the height to which the contraption can launch the object (ignoring the weight of the messenger ... being virtual, their weight is measured in photons), in **meters** rounded to the nearest one hundredth. Print the name of the contraption and how high it will launch our messenger using the format given below. If it will reach at least 25.00 meters, print afterwards: SUCCESS, else print: SPLAT. *However* there is a roof over the tunnel in the cliff the Allies are aiming for starting **50 meters up**. If the messenger will be launched *higher* than 50 meters, print OUCH (because they will hit the roof).

```
GIANT-SLINGSHOT will launch the messenger 5.16 meters high, SPLAT!
```

Discussion

$$\text{height} = \frac{\text{speed}^2}{2(\text{gravity})} = \frac{\text{speed}^2}{2\left(\frac{9.805\text{m}}{\text{s}^2}\right)}$$

- 1 meter is equal to 3.28 feet for the purposes of this problem

Because the units for the acceleration of gravity are given in meters/second, and because your output needs to be in meters, your first step needs to be converting your input rate of speed to meters/second (if it isn't already in meters/second). After that simply square the speed and then divide by 2 times the acceleration of gravity (9.805m/s^2) to get the maximum vertical distance the contraption will launch our brave messenger.

In this example, 1980.00 feet / minute converts to 33 feet / second. Converting feet to meters (using the conversion of 3.28 m / f given above) then gives us 10.060975609756097560975609756098 meters / second (don't round anything until printing your final answer.)

Squaring that will give us 101.22323022010707911957168352171 (m/s)². We then divide that by $2*(9.805\text{m/s}^2)$.

That gives us a distance of 5.1618169413619112248634208833102 meters traveled, which we will round to the nearest hundredth giving us 5.16, which is TOO SHORT, so our messenger will hit the cliff wall, so we print SPLAT! after our output.

Reminder: have you run your solution against **all** of the student data sets?

Additional Examples

Input 1	Output 1	Input 2	Output 2
CANNON 432.00 KILOMETERS PER HOUR	CANNON will launch the messenger 734.32 meters high, OUCH!	BIG-PAPER-AIRPLANE 25.64 YARDS PER SECOND	BIG-PAPER-AIRPLANE will launch the messenger 28.04 meters high, SUCCESS!

Trapped by a river and racing against time, our fearless heroes need to quickly cross it in order to stop mom from placing the wrong pizza order. (Funny story, turns out Greg was only joking about the anchovies).

Luckily, our heroes have found a ramp on their side of the river (what could go wrong?). Help them figure out if they can jump the river (stunts performed on closed course by trained professionals only).



Input

You will receive 4 lines of information:

```
Motorcycle
100
30.0
780.1
```

- Line 1: The name of the vehicle
- Line 2: The length of the ramp (in meters, always a whole 32-bit integer)
- Line 3: The acceleration rate of the vehicle (in meters/second-squared, floating point decimal of max size 2147483647.0)
- Line 4: The width of the river (in meters, floating point decimal of max size 2147483647.0)

Output

Using that information, calculate the horizontal speed (rounded to the nearest hundredth) the vehicle will be going when it runs out of ramp, and then use that to calculate how much horizontal distance (rounded to the nearest tenth) your vehicle will be able to cover (formulas in the discussion section) and output the results of your ramp jumping!

```
Motorcycle will reach a speed of 77.46 m/s on a 100 meter ramp, crossing 611.9 of 780.1 meters,
SPLASH!
```

If the distance which can be covered is:

- < the width of the river minus 5 meters, add **SPLASH!** to your output
- >= river-width minus 5 meters **AND** <= river-width, add **BARELY MADE IT!**
- > river-width, add **LIKE A BOSS!**

Discussion

This will be a multi-equation calculation using the following data:

- ramp_1 = Ramp length
- rate_1 = Vehicle acceleration rate (Note, in this problem, no vehicle has a maximum speed, they can all accelerate indefinitely, because *SCIENCE!*)

You will first need to figure out at what time the vehicle will run out of ramp if accelerating at the constant rate given in the input. Solve for that time (in seconds) using:

$$\text{time}_1 \text{ (until ramp runs out)} = \sqrt{\frac{2(\text{ramp}_1)}{(\text{rate}_1)}}$$

Once you have that time, plug that into the following formula to solve for the rate of speed the vehicle will be traveling at the moment it runs out of ramp:

$$\text{speed}_1 \text{ (at end of ramp)} = (\text{time}_1)(\text{rate}_1)$$

Now, with those 2 pieces of key information you can calculate the distance the vehicle will be able to cover when launched diagonally to cross the river (under ideal conditions):

$$\text{distance} = \frac{(\text{speed}_1)^2}{9.805}$$

So, in our example, given a Motorcycle using a 100 meter long ramp, with a constant acceleration rate of 30m/s, we will get: * $\text{time}_1 = 2.58\text{s}$ until it reaches the end of the ramp * $\text{speed}_1 = 77.46\text{m/s}$ rate of speed when it leaves the ramp * 257.8 meters distance traveled

Disclaimer We are aware that this isn't fully accurate according to physics -- in the universe of this problem, the equations given are correct, and all that you need.

Reminder: have you run your solution against **all** of the student data sets?

Additional Examples

Input 1	Output 1	Input 2	Output 2
Rollerskates 200 5.0 50.9	Rollerskates will reach a speed of 44.72 m/s on a 200 meter ramp, crossing 204.0 of 50.9 meters, LIKE A BOSS!	GiantHamsterBall 301 2.6 162.0	GiantHamsterBall will reach a speed of 39.56 m/s on a 301 meter ramp, crossing 159.6 of 162.0 meters, BARELY MADE IT!

The Mask ate a block of dynamite to save Tina during Dorian's rampage -- now he has a taste for it. Help the Mask figure out how much dynamite he can eat without being destroyed.



Input

You will receive on a line 3 integers (or fractions) separated by spaces. The first number will be the number of sticks of dynamite. The second will be the *size* of the stick (1/4, 1/3, 1/2, 1, 2 or 3). The third number will be the tensile limit (in Megajoules (MJ)) the Mask can tolerate at that moment, due to being tired, having suffered other damage, being freshly out of bed, etc.

```
8 1 82
```

Output

Determine if the Mask can survive the energy output from the blast if he eats the dynamite. See discussion section for formula. Print the amount of explosive force the dynamite will produce (rounded to 2 decimal places), a space then if the force is $\leq$ the tensile limit the Mask can tolerate, print "the Mask can eat it!", else print "the Mask should not eat it!"

```
27.00 the Mask can eat it!
```

Discussion

To determine the Megajoules (MJ) of explosive force, you will need the following information:

- One (1) **whole** stick of dynamite weighs **0.45 kilograms (kg)**.
- **7.5MJ** of explosive force are released when **1kg** of dynamite explodes.

Reminder: have you run your solution against **all** of the student data sets?

Additional Examples

Input 1	Output 1	Input 2	Output 2
3 2 20	20.25 the Mask should not eat it!	10 1/4 9	8.44 the Mask can eat it!

Princess Celestia wants everypony to line up for the start of her Summer-eve ball.

She has asked you, Twilight Sparkle, to sort the ponies alphabetically but with royalty in front. Royal ponies have gemstones in their names.



Input

You will receive a list of ponies numbering anywhere from 5 to 30 ponies, with 1 per line. The end of input will be marked with END on a line by itself. Names are guaranteed to be no longer than 80 characters in length, and to only contain letters and spaces.

```
Fluttershy
Applejack
Rubyanne
Rainbow Dash
Rarity
Pinkie Pie
Emerald Sunshine
Spike the Dragon
Misty Sapphire
END
```

Output

Sort and list the ponies alphabetically in ascending order ('A' "first"), ignoring case. However, any pony with a gemstone in their name must be placed at the "top" of the list (before the "A's" start) in the gemstone order given in the Discussion section below.

```
Misty Sapphire
Emerald Sunshine
Applejack
Fluttershy
Pinkie Pie
Rainbow Dash
Rarity
Rubyanne
Spike the Dragon
```


Discussion

The Pony ranking (in Ascending order) for gemstones is as follows:

```
Lapis
Topaz
Tourmaline
Sapphire
Peridot
Ruby
Pearl
Emerald
Diamond
Aquamarine
Amethyst
Garnet
```

Note, you are guaranteed that you will not have to deal with any gemstones not listed above. If multiple gems are listed in the same name, sort by whichever gem has "highest priority" (e.g. a pony named 'Garnet Lapis' would be listed before a pony named 'Topaz Sapphire,' because Lapis has the highest priority).

In the case of equal ranking on gemstones, output in alpha order by total name (not just the gemstones), only sort by highest priority gemstones, after that by alpha. (Example, given the names: Lapis Topaz and Amethyst Lapis, they should be printed in order as: Amethyst Lapis then Lapis Topaz. That is because both names have the highest ranked gemstone (Lapis), so we simply sort them in ascending order alphabetically after their ranking has been established. We would not list Lapis Topaz before Amethyst Lapis because Topaz has a higher ranking over Amethyst. Stop comparing gemstone ranking after determining the *highest* rank of the gems in the name.)

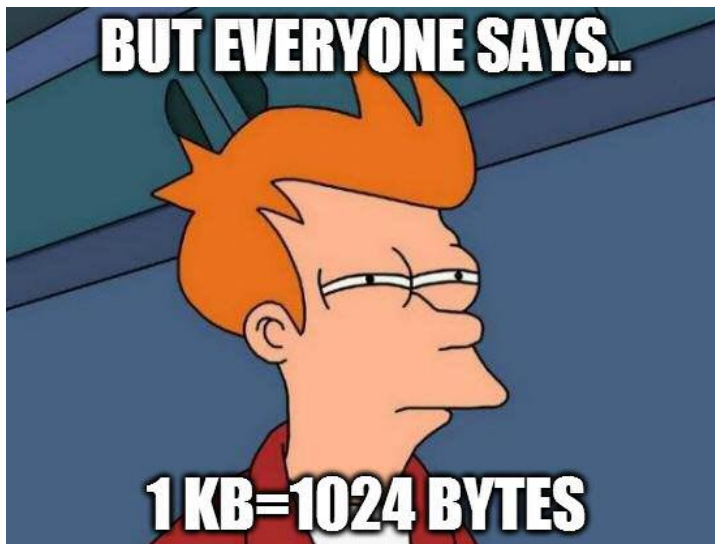
You are guaranteed that there will be no *ties* in priority for gemstone ranking. You are also guaranteed that you will not encounter hyphenated names like Ruby-Sue. However, if a gemstone name *happened* to be found as part of a name (like *Rubyanne*) you can safely treat that as just another name to alphabetize. Gemstone names have to stand on their own (separated by spaces, or the entirety of the name) to be treated royally.

Reminder: have you run your solution against **all** of the student data sets?

Additional Examples

Input 1	Output 1	Input 2	Output 2
Cherilee	Sapphire Shores	Photo Finish	Pearl Peridot
Sapphire Shores	Cherilee	Discord	Princess Pearl
Tirek	Grubber	Spitfire	Discord
Songbird Serenade	Songbird Serenade	Soarine	Granny Smith
Tempest Shadow	Tempest Shadow	Pearl Peridot	Photo Finish
Grubber	Tirek	Granny Smith	Soarine
END		Princess Pearl	Spitfire
		END	

Fry is suspicious that the hard drive he just bought for his computer *said* 1EB on the box, but when he plugged it into his computer the operating system says it only has 931PB of space. Leela says that's because hard drive marketing uses base-10 to calculate space, but computer science (and operating systems) use base-2 (and always have). So, using base-10, 1 Exabyte (EB) would be 10^{18} (1,000,000,000,000,000,000) bytes. But in base-2 it would be 2^{60} (1,152,921,504,606,846,976) bytes. Most humans use base-10 when counting, so there is confusion. (Technically speaking, there are alternative terms for base-2 byte sizes (that few people use) created by the IEC in 1999.) Help Leela explain it to Fry by writing a program that will take storage space given in base-10, and convert it to base-2 using the tables below for reference.



Base-10 (SI units)		Base-2 (Binary)	
8 Bits	= 1 Byte (B)	8 Bits	= 1 Byte (B)
1000 B	= 1 Kilobyte (KB)	1024 B	= 1 Kibibyte (KiB)
1000 KB	= 1 Megabyte (MB)	1024 KiB	= 1 Mebibyte (MiB)
1000 MB	= 1 Gigabyte (GB)	1024 MiB	= 1 Gibibyte (GiB)
1000 GB	= 1 Terabyte (TB)	1024 GiB	= 1 Tebibyte (TiB)
1000 TB	= 1 Petabyte (PB)	1024 TiB	= 1 Pebibyte (PiB)
1000 PB	= 1 Exabyte (EB)	1024 PiB	= 1 Exbibyte (EiB)
1000 EB	= 1 Zettabyte (ZB)	1024 EiB	= 1 Zebibyte (ZiB)

Input

You will receive a computer hard drive size as a whole integer, a space, then a 2-letter size code reported in Base-10 SI Units from the marketing text on the box. Your program should then convert to the base-2 Binary size the hard drive will show as available space in the operating system **rounded** to the nearest **2 decimal places** in the largest binary size you can express a whole number in (e.g. do not write 1030 MiB, write 1.01 GiB)

1 TB

Output

Your output should convert the size given in base-10 to base-2 units which will be reported in the operating system, **rounded** to 2 decimal places. Make sure Leela's program outputs the base-2 Binary 3 letter code for her program's output, to help Fry understand the differences.

931.32 GiB

Discussion

Reminder: have you run your solution against **all** of the student data sets?

Additional Examples

Input 1	Output 1	Input 2	Output 2	Input 3	Output 3
1 ZB	867.36 EiB	752 MB	717.16 MiB	1105 KB	1.03 MiB

Leia, Luke, Han and Chewie are trapped in the Garbage Masher on the detention level of the Death Star, and the masher's walls are coming in!

Luke must call R2-D2 and C-3PO for help! The walls move in at 60cm per minute. The walls start 5 meters apart. You need to figure out if everyone makes it out of the masher or disaster strikes. There must be at least 20cm remaining between the walls for everyone to make it out a little thinner, but okay.

It will take C-3PO 40 seconds to successfully tell R2-D2 to shut down the correct Garbage Masher. May The Force Be With You!



Input

You will receive 2 sets of data on one (and only one) line in your input. The data will be the time (HH:MM:SS in 24 hour notation) the walls started moving, a space, then the time Luke remembers his communicator and calls the droids for help. You are guaranteed that none of the input times will roll over to the next day (e.g. you won't see input like this: 23:59:19 00:10:16)

```
13:05:01 13:07:08
```

Output

Your program must output 3 pieces of information:

1. The remaining space between the walls (in meters, rounded to 2 decimal places)
2. A schematic of the walls as a digital rendering (for the Imperial archives, of course). Use the pipe symbol (|) to represent the walls in your digital rendering (one pipe symbol is roughly equal to 2 meters in height, and the walls of the masher are 12 meters tall. It also just so happens that 1 period (.) character on a screen is equal to 20cm. The Emperor was obviously planning ahead. The walls will start 5 meters apart. In your digital rendering show how close the walls are when the droids will have stopped them (rounded down to the nearest 20cm increment (e.g. if there would be 25cm left, output one period, if there would be 1m 19cm left, output 5 periods)).
3. If there will be less than 20cm of space between the walls when they stop, print on a new line: CURSE MY METAL BODY, I WASN'T FAST ENOUGH! If there are **exactly** 20cm remaining between the walls, print on a new line: THIS IS SOME RESCUE! Else, print: THE FORCE WAS WITH YOU

```
3.33
```

```
|
|
|
|
|
|
|.....|
```

```
THE FORCE WAS WITH YOU
```

Discussion

2 minutes, 7 seconds elapse from when the walls start moving until Luke remembers to call. At 60cm/minute that costs them 127cm. The 40 seconds the droids take to act costs an additional 40cm. This brings the walls in a total of 167cm, or 1.67 meters. Subtracting 167cm from the initial 500cm (5m) wall separation gives 333cm, or 3.33m. Rounding down to 320cm, you must print 16 period characters in your digital rendering for the Imperial archives (remember to round **down** to the nearest 20cm!). (Note, the walls will stop at zero, the distance between them cannot go negative.)

Reminder: have you run your solution against **all** of the student data sets?

Additional Examples

Input 1	Output 1	Input 2	Output 2
08:59:05 09:09:59	0.00 CURSE MY METAL BODY, I WASN'T FAST ENOUGH!	22:20:00 22:27:20	0.20 . THIS IS SOME RESCUE!

Doc Brown needs to help Marty McFly get back to the future. To do that he needs to generate 1.21 gigawatts (GW) of electricity in one hour.

Unfortunately, that amount of energy isn't readily available in the 1950s. Help Doc Brown brainstorm ways he can generate 1.21 gigawatts of electricity (excluding lightning, because, you know ... *it's been done*) over the span of an hour. (Note: a gigawatt is 1 billion watts.)



Input

You will receive the following information in your data file, one per line:

- The method name of generating electricity
- The number of people required to run the method
- The amount of time the method can be run given Doc's resources (in minutes)
- The startup time it takes before electricity is produced (in seconds)
- The power output of the method (in watts/hour)
- The number of people Doc Brown could borrow, beg, pay or cajole into helping him generate electricity for an hour

Here's the example input:

```
Nuclear power plant (don't ask)
8672
60
3599
4392000000000.00
8672
```

Output

Output the name of the method and the total amount of watts/second the method can generate. Power can only be generated if people can still run the method, and time hasn't run out. Output the total rounded to 2 decimal places. **Doc needs to power the time machine for a full 1 second.** If the method will produce enough electricity, output on a new line: MARTY CAN MAKE IT! Else, output: WHOA, HEAVY!

```
Nuclear power plant (don't ask) can generate 1220000000.00 watts/second
MARTY CAN MAKE IT!
```


Discussion

Power output for a method of generating electricity are given in watts per hour. But Doc Brown only needs to generate and *sustain* the 1.21 gigawatt for **1 second**. So, as long as the method of generating power doesn't take too long to start up (and there are enough people to run it), all you really need to look at is the amount of watts the method can produce per hour, and translate that into watts per second (which is all Doc cares about).

However, if there aren't enough people to run the method, it won't work. Also, if it takes longer to start up than the amount of time we have, it won't work.

So, in this example: we have *just enough* people to run the nuclear plant, and we can run it for a total of 1 hour (60 minutes). It takes 3,599 seconds to start up (1 second shy of 1 hour), so we will have *exactly* 1 second of output before we have to stop it (hey, nuclear power wasn't exactly stable in the 1950s, this was experimental), but luckily, the plant outputs over 4 trillion terawatts of power/hour! That equals 1.22 gigawatts/second, so it'll work (barely).

Reminder: have you run your solution against **all** of the student data sets?

Additional Examples

Input 1	Output 1	Input 2	Output 2	Input 3	Output 3
Bicycle 1 60 3 100.0 5000	Bicycle can generate 0.03 watts/second WHOA, HEAVY!	Area 51 Spaceship 137 31 1959 999999999999.99 140	Area 51 Spaceship can generate 0.00 watts/second WHOA, HEAVY!	Wind Turbine Farm 500 60 0 499998888888.35 350	Wind Turbine Farm can generate 0.00 watts/second WHOA, HEAVY!

In Input 1, it takes 1 person to run a bicycle generator, and they can do it for an hour. It takes 3 seconds before the method starts generating power. So, we have enough people (5000, 4999 more than fit on a bicycle), and the startup time and running time allow 1-second of sustained power, so we can convert watts/hour into the watts/second the method can produce and output it.

In Input 2, while the power output from the spaceship would be **huge**, Doc doesn't have the ability to run it long enough to generate electricity, as it will take more time to startup (1959 sec) than he can run it (1860 sec).

In Input 3, Doc doesn't have enough people (only 350, and need 500) to run the Wind Turbine Farm, so again, it can't generate any sustained electricity.

We need a squad that will be able to beat Thanos's Army.

Each Avenger has a certain overall starting strength. Avengers also have synergy with each other, and the synergy has a multiplying effect. In the table, the column below each avenger lists their multipliers. For example, in Thor's column, we see Thor receives 1.7 synergy from Iron Man, 1.9 synergy from Hulk, 1.2 synergy from Spider Man and 1.4 synergy from Captain America. So, if Thor has a starting power of 50, his power with Iron Man, Hulk, Spider Man and Captain America becomes $(50)(1.7)(1.9)(1.2)(1.4) = 271.32$. We do the same with all Avengers on the team. After we apply synergy to each Avenger, we sum the powers. For instance, if Thor has 271.32, Iron Man 63.2, Hulk 10.2, Black Widow 14.1 and Hawkeye 13.1 the sum would be 371.92. We then round to the nearest integer to find the Team Power: 372.

A different example: If Iron Man's starting power is 10, and he is partnered with Captain America, Thor, Black Widow and Hawkeye, Iron Man's column shows his starting power is multiplied by 1.7, 1.4, 1.7 and 1.7, resulting in 68.782. We would then sum Iron Man and all 4 other Avenger powers and round.

Synergy Map:

The number 1 is used as a placeholder for an Avenger's self, because 1 multiplied by one's own power does not modify one's own power.

	Iron Man	Hulk	Spider Man	Captain America	Thor	Black Widow	Hawkeye
Iron Man synergy multiplier	1	1.2	2.0	1.6	1.7	1.7	1.7
Hulk synergy multiplier	1.4	1	1.3	1.5	1.9	1.9	1.6
Spider Man synergy multiplier	1.7	1.5	1	1.7	1.2	1.4	1.1
Captain America synergy multiplier	1.7	1.7	1.7	1	1.4	1.3	1.8
Thor synergy multiplier	1.4	1.8	1.2	1.6	1	1.4	1.3
Black Widow synergy multiplier	1.7	1.4	1.3	1.8	1.4	1	2.0
Hawkeye synergy multiplier	1.7	1.6	1.1	1.8	1.3	2.0	1

Input

Input is a single line of text of the form "A P, A P, A P, A P, A P, TA P". Read the team in with each Avenger and their Power. Each team member is separated by a comma. There will be 5 team members, and the end of the line will be power of Thanos's Army ("TA"). Powers are integers.

```
Thor 16, Hulk 100, Spider Man 1, Black Widow 162, Hawkeye 14, TA 1902
```

Output

Apply all the synergy multipliers to each Avenger's power, sum the powers up and round, and determine if the Avengers will defeat Thanos's Army. Print "Draw", "Avengers win!" or "Thanos's Army wins!" and then a space, then the power totals dash-separated with the higher number on the left.

```
Avengers win! 1944-1902
```

Additional Examples

Input 1	Output 1
Thor 72, Hulk 64, Black Widow 88, Captain America 70, Spider Man 95, TA 2001	Draw 2001-2001

Discussion

With the student datasets, there is a ThanosTableData.txt file with a comma-separated matrix of the synergy map. You may copy this data into your source code if it helps you. **THIS FILE WILL NOT BE PRESENT WHEN JUDGES TEST YOUR PROGRAM**, so don't read it as an input file in your program.

Bunny wants to build a city out of pancakes but wants a blueprint of the city built.

He will give you the height of the buildings plus its address (a single letter). Sort the building left to right smallest to tallest. If multiple buildings have the same height, sort in alphabetical (ascending A-Z) order.

As a special bonus, top each building with a cherry on top "~".



Input

First line of input contains the number of buildings in the blueprint. The following lines will contain the height of a building followed by a space and the letter representing its address. The height integer will contain a leading zero if it is less than 10.

```
5
02 A
03 B
02 C
01 E
05 D
```

Output

Print the pancake towers from left to right, smallest to tallest and under each tower print its address. Don't forget the "~" cherry on top of each pancake tower!

```
~
*
~*
~**
~***
~****
*****
EACBD
```

Dr. Evil has stolen Austin Powers' mojo! He's hidden it where he thinks Austin will never find it. Help Austin find his Mojo!

Input

You will receive up to 25 lines of UPPERCASE ASCII alphabet characters (A-Z). Each line can contain up to 25 ASCII characters as well. (With a maximum problem space of 25x25. The rows and columns will always form a rectangle -- they will never be uneven) Hidden somewhere in the lines of characters will be the word "MOJO".

```
GKJKSMBJGS
KJGSDOHGBH
TQBCNJLOED
SDJHJODKLK
```



The word may be written: horizontally left to right, or right to left. It may also be written vertically top to bottom or bottom to top. It may also be written as a square (Dr. Evil knows Austin disapproves of squares, you see...) If it is written as a square it will always be written as:

```
MO
JO
```

Output

Starting with coordinate (X=0,Y=0) in the upper left corner (and the max coordinate in the lower right corner), find Austin's MOJO and tell him where to find it by printing out the coordinates for each letter (in order, M-O-J-O) as follows:

```
M: 5,0
O: 5,1
J: 5,2
O: 5,3
```

Discussion

Dr. Evil couldn't figure out how to hide Austin's mojo diagonally, so you don't need to worry about helping Austin find it diagonally.

Reminder: have you run your solution against **all** of the student data sets?

Additional Examples

Input 1	Output 1	Input 2	Output 2	Input 3	Output 3	Input 4	Output 4
QWER	M: 0,2	MNBVCX	M: 5,4	QAZWSXE	M: 5,1	PMOKM	M: 1,0
TYUI	O: 1,2	ZLKJHO	O: 5,3	DCOJOMG	O: 4,1	IJOUH	O: 2,0
MOJO	J: 2,2	FDSAPJ	J: 5,2	BYHNUJM	J: 3,1	BYGVT	J: 1,1
DFGH	O: 3,2	IUYTRO	O: 5,1	UJKILOP	O: 2,1		O: 2,1
		WQMNB					

Remy needs help! He made his Ratatouille for Anton Ego again and now Ego wants the recipe. Ego doesn't have the time to convert measurements so he needs a recipe for the needed servings that he wants to make.

Anton needs the recipe to be in the easiest form possible. He likes to have large get-togethers and he needs to be able to make the recipe quickly. Please convert each ingredient to use the largest measurement possible but greater than 1. The recipe given is a single serving.

Input

You will receive the same recipe (for 1 serving) in every dataset, but with a different **new serving amount** as the first line. Ingredients for one serving are listed one per line, and split up into three sections (WET, DRY, WEIGHTED). All numbers given are floating point decimals. Units and ingredients are always written as singular, never plural (**cup** not *cups*, **eggplant**, not *eggplants*). Each line will be in the format of: *measurement space unit space ingredient*



```
4.00
WET INGREDIENTS
2.00 cup tomato-sauce
2.25 tablespoon olive-oil
DRY INGREDIENTS
0.50 teaspoon salt
0.25 teaspoon black-pepper
1.00 cup onion
WEIGHTED INGREDIENTS
4.00 ounce eggplant
```

Output

Output should be the same recipe as the input, but with the measurements and units changed to match the new serving size. **Note** the new serving size can be less than 1 (e.g. 0.75, 0.50, 0.33, 0.25, etc.), but the new **serving size will never be zero**. All number output should be rounded to the nearest hundredths (0.00). (Due to differences in rounding libraries, we will accept answers +/- 0.01 from what we are expecting.)

```
WET INGREDIENTS
2.00 quart tomato-sauce
4.50 ounce olive-oil
DRY INGREDIENTS
2.00 teaspoon salt
1.00 teaspoon black-pepper
4.00 cup onion
WEIGHTED INGREDIENTS
1.00 pound eggplant
```


Discussion

Use the largest units you can use which keeps your output in whole numbers. E.G. 15 ounce would stay 15 ounce not 0.9375 pound. 17 ounce should be converted to 1.06 pound. 5 cup to 1.25 quart, etc.

Use the conversions given below to move up/down units as needed for wet, dry and weighted ingredients. **The only data that should change in your output from the input are the measurements and units.** The ingredient names should stay the same, as should the SECTION HEADERS in all caps.

WET:

- 1 gallon = 4 quart
- 1 quart = 2 pint
- 1 pint = 2 cup
- 1 cup = 8 ounce
- 1 ounce = 2 tablespoon
- 1 tablespoon = 3 teaspoon

DRY:

- 1 cup = 16 tablespoon
- 1 tablespoon = 3 teaspoon

WEIGHTED:

- 1 pound = 16 ounce

Additional Examples

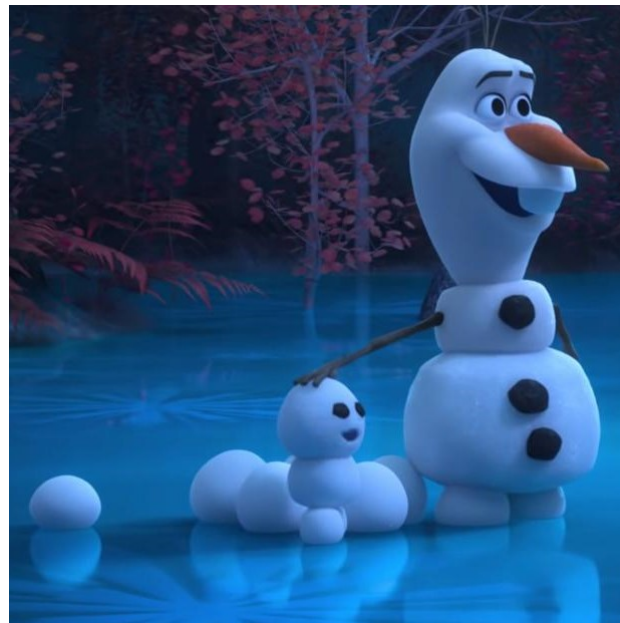
Input 1	Output 1
0.50 WET INGREDIENTS 2.00 cup tomato-sauce 2.25 tablespoon olive-oil DRY INGREDIENTS 0.50 teaspoon salt 0.25 teaspoon black-pepper 1.00 cup onion WEIGHTED INGREDIENTS 4.00 ounce eggplant	WET INGREDIENTS 1.00 cup tomato-sauce 1.12 tablespoon olive-oil DRY INGREDIENTS 0.25 teaspoon salt 0.12 teaspoon black-pepper 8.00 tablespoon onion WEIGHTED INGREDIENTS 2.00 ounce eggplant

Input 2	Output 2
1000.00 WET INGREDIENTS 2.00 cup tomato-sauce 2.25 tablespoon olive-oil DRY INGREDIENTS 0.50 teaspoon salt 0.25 teaspoon black-pepper 1.00 cup onion WEIGHTED INGREDIENTS 4.00 ounce eggplant	WET INGREDIENTS 125.00 gallon tomato-sauce 8.79 gallon olive-oil DRY INGREDIENTS 10.42 cup salt 5.21 cup black-pepper 1000.00 cup onion WEIGHTED INGREDIENTS 250.00 pound eggplant

Olaf wants to build a snowman, but he wants your help to create the right size snowballs.

Please create the snowman using the following guidelines:

- Each snowball will be "circular" (more diamond-shaped, really)
- The total height of the snowball is the size plus an extra 2 for the top and the bottom layer
- Snowball sizes will only be odd numbers
- The width at the top and the bottom of a snowball should equal the given size.
- All of the snowballs need to be centered on the largest snowball
 - **Hint:** The leftmost part of the top snowball will be the largest size given minus size/2.
- The largest snowball should have no padding to the left at the center (largest) part of the snowball.



Input

The first line will contain the number of snowballs that the snowman is made out of. The lines following that are snowball sizes in order from top to bottom ranging from 1 to 29. Each snowball size is given on its own line.

```
4
1
5
7
3
```

Output

The snowman blueprint for Olaf based on the snowball sizes and rules.

```
#
# #
#
#####
#   #
#   #
#   #
#   #
#   #
#####
#####
#   #
#   #
#   #
#   #
#   #
#   #
#####
###
#   #
#   #
#   #
###
```

Additional Examples

Input 1	Output 1	Input 2	Output 2
3 7 5 5	<pre> ##### # # # # # # # # # # # # ##### ##### # # # # # # # # # # ##### ##### # # # # # # # # ##### </pre>	4 3 7 1 5	<pre> ### # # # # # # ### ##### # # # # # # # # # # ##### # # # # ##### # # # # # # # # ##### </pre>

Johnny is the best *hacker* the world has ever seen (according to Johnny).

He is 100% positive that **nobody** could crack his 3L1+3 *encryption* he's using to protect all of his secret notes. Johnny is so confident that he's bet you \$50 you can't crack his 3L1+3 base-74 (that's **10 better** than base-64!) *encryption* method.

Show Johnny why there is a difference between 1-way hash **encryption** and 2-way hash **encoding** that can be easily reversed *without a password*.



Input

You will receive 2 lines of input. On the first line will be a base-74 *encoded* line of text That line will be no longer than 128 ASCII characters. On the second line you will receive the base-74 hashing lookup table (always of length 84) Johnny used in his algorithm (this can change for every dataset).

```
RZqG~Suw
~Tvs{0)2$"tGQ[*1uq6!~%IWR]PSZ<?5@VmLzJFM}K8EpoB&>1An3,0C#7Xw9HUrx.4D(ky/NY
```

Output

Use Johnny's algorithm given below to **REVERSE** the encoding (**DECODE**) and print out the original message before it was base-74 encoded.

```
ABC123
```

Discussion

Johnny's base-74 algorithm:

1. Take the ASCII ordinal value of each character in the original string (e.g. 'A' is 65, 'B' is 66, 'C' is 67, etc...)
2. Calculate the 8-bit binary equivalent of the ASCII values and group them as a large string (e.g. 65 becomes 01000001, 66 becomes 01000010, 67 becomes 01000011, etc...)
3. Convert the 8-bit grouped string into chunks of 6 bits by re-grouping the digits (e.g. 010000010100001001000011... breaks down, working from left to right, into new groups of 6 bits as: 010000 010100 001001 000011...)
4. Convert the 6-bit binary groups to their respective decimal values **then add +8** to that value. (e.g. 010000 is 16 in decimal, add 8 to that and you get **24**. 010100 is 20, +8 == **28**. 001001 is 9, +8 == **17**. 000011 is 3, +8 == **11**, etc...)
5. Using the base-74 lookup table from line 2 of the input, assign the respective base-74 character for each calculated decimal value, using that as an index to pick a character from the table. (e.g. index 24 in the first base74 example lookup table is character 'R'. 28 is 'Z'. 17 is 'q'. 11 is 'G', etc...)

(Note, for those of you who know how base-64 works, and you're wondering about the padding-terminal characters, Johnny is very careful to craft messages that never need such padding. So you don't need to worry about it.)

Reminder: have you run your solution against **all** of the student data sets?

Additional Examples

Input 1

```
^a}>c00tA&hjc9dqBvDmc9)nE76eD&toCq6~D,^qcb@/) #6g(*?x@aL.  
GHIJ1234567890abcdefghijklmnopqrstuvwxyzKLMNOPQRSTUVWXYZ
```

Output 1

```
The Quick Brown Fox Jumps Over the LazyDog
```

Input 2

```
88uqk8uLjz0pm(~Jn8uEkN7*kvuGiruq2zMGiz~Jkru%#!AGlrM1i!Q*1(yD  
abcdefgh}uvwxyz12345$%^&67890~!@#ijklmnopqrst*(){ABCDEFGHIJKLMN  
OPQRSTUVWXYZ [] <> , . / ? "
```

Output 2

```
I am the very model of a modern Major-General
```


Shrek isn't happy that Lord Farquaad had his goons drop off all of the fairy tail creatures at his swamp, and Donkey isn't making it any easier. Help Shrek chase Donkey around his swamp by looking at the path Donkey is running, and helping to choose a path Shrek can take to catch him!

Input

You will receive 10 lines of input, 3 for Donkey followed by 7 for Shrek. Lines will use formatting of:

NAME

Y a b c

N.NNN N.NNN +/-

Where:

- **NAME** is either SHREK or DONKEY, indicating the start of their section
- **Y** is the letter code for the equation (always UPPER CASE)
- **a** is the x-squared term coefficient of a quadratic equation (positive or negative integer)
- **b** is the x-term coefficient (positive or negative integer)
- **c** is the y-absolute-term (positive or negative integer)
- **N.NNN N.NNN** are the x- and y-coordinates of the starting point (positive or negative, with 3 decimal places).
- **'+'** means they are traveling in a positive x-direction, **'-'** means a negative x-direction

Note: Donkey has only one path equation. Shrek has three.

DONKEY

Y 1 0 0

0.000 0.000 +

SHREK

A 1 0 2

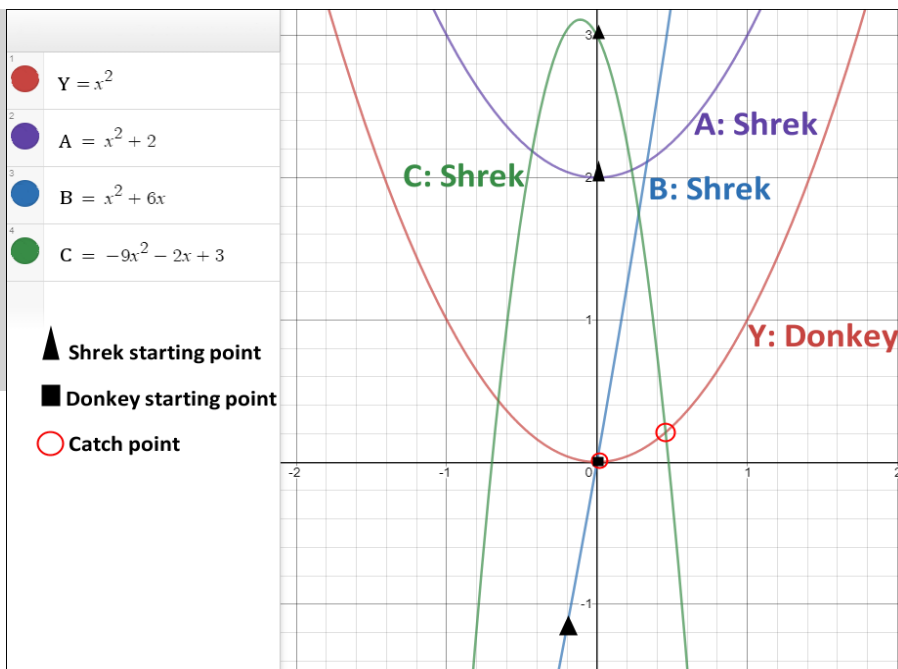
0.000 2.000 +

B 1 6 0

-0.200 -1.160 +

C -9 -2 3

0.000 3.000 +



Output

For each of Shrek's three paths, find if Shrek will **catch** (intersect) Donkey in the *direction* Donkey is running. If Shrek's path won't intersect Donkey's in the direction they are traveling, print MISS! If Shrek's path will intersect Donkey's path, calculate the distance between the starting and intersection point. **Do not consider intersection points in directions Shrek/Donkey are not traveling.** Print the path letter and the closest intersection point (x,y), rounded to 3 decimal places. **Only round your printing AFTER calculating the distance.** After evaluating all three paths, choose the path with the *smallest* distance from Shrek's starting point to the intersection point with Donkey, and print out a fourth line with the equation letter (A, B, or C) and the words "CATCHES DONKEY N.NNN METERS AWAY" (where N.NNN is the distance). If no intersections are found in any path, print "DONKEY GETS AWAY!"

```
A MISS!
B (0.000,0.000)
C (0.457,0.209)
B CATCHES DONKEY 1.177 METERS AWAY
```

Discussion

This is an intersection-of-equations problem. If you haven't had Algebra level one yet (or have forgotten it), you will need the quadratic formula to solve for the intersection points. To determine intersection of two equations, subtract one equation from another (subtract the 'a', 'b', 'c' terms).

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

- If the new 'a' is zero, the new equation is "bx+c", so the only intersection is x= (-c/b).
- But if the new 'b' is also zero, that is undefined (so there's no intersection).
- If the new 'a' is not zero, plug the coefficients into the quadratic formula.

The quadratic formula will usually give you two X-values. The X results you get can then be substituted into one of the two original equations to get the values for Y, which would then give you a complete set of coordinates of intersection. *But remember to pay attention to which direction they are both traveling!*

To get the distance between Shrek's starting point until he catches Donkey, take the x/y-coordinates from Shrek's starting point and the x/y-coordinates from the intersection with Donkey's path, and use the distance formula shown.

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

In path A, we don't intersect. Subtracting Y from A, we get the equation "2" (a=0, b=0, c=2). There is no value of x which would make that equation be zero, so there are no intersections.

In path B, subtracting Y from B, we get "6x", so the intersection is at x=(-0/6)=0. Shrek starts at (-0.200,-1.160) and path B intersects Donkey's at (0,0) (which is Donkey's starting point -- apparently Shrek snuck up on him). So, to get the distance traveled find $\text{sqrt}((0.2 * 0.2) + (1.16 * 1.16)) = 1.177$ meters.

In path C, subtracting Y from C, we get "-10x^2 -2x +3" (a=-10, b=-2, c=3). Using the quadratic formula gives two possible X values: (0.457) and (-.657). Since Shrek and Donkey both start at 0 and are moving positive, we don't consider the negative X-value and only use 0.457. Shrek starts at (0,3) and that path intersects Donkey's at (0.457,0.209). So, to get the distance traveled we use (x2-x1): |0.457 - 0| and (y2-y1): |0.209 - 3| which will give us a distance of $\text{sqrt}((0.457 * 0.457) + (2.791 * 2.791)) = 2.828$ meters. (Remember, you need to round your intersection coordinates when you print them, but don't round them before using them to calculate distance! Only round numbers being printed to output.)

Of the two distances we have where Shrek caught Donkey, path B gives the shortest distance, so that is the distance to print on line 4 of the output.

Reminder: have you run your solution against **all** of the student data sets?

Frodo, Aragorn, Gimli, Legolas and the rest of the Fellowship of the Ring are trapped in the mines of Moria with a Balrog hot on their tails (heh).

The Dwarves of the Misty Mountains are clever and have engineered the halls and tunnels under their mountain into a confusing labyrinth to help keep greedy dragons and invading orcs out.

Gimli has several maps from his cousin Balin which give the exact location of timed doors along the path the fellowship must take to escape the mines.

The day may come when the fellowship fails, but with your help, *it will not be this day!*

Input

On the first line of input you will receive 3 integers, separated by spaces. The integers will be the WIDTH of the map you will receive, the HEIGHT of the map, and the number of DOORS in the map. After the map, rendered in ASCII characters, will be a list of the specific locations and types of DOORS in the map, in the following format:

C N X Y

Where:

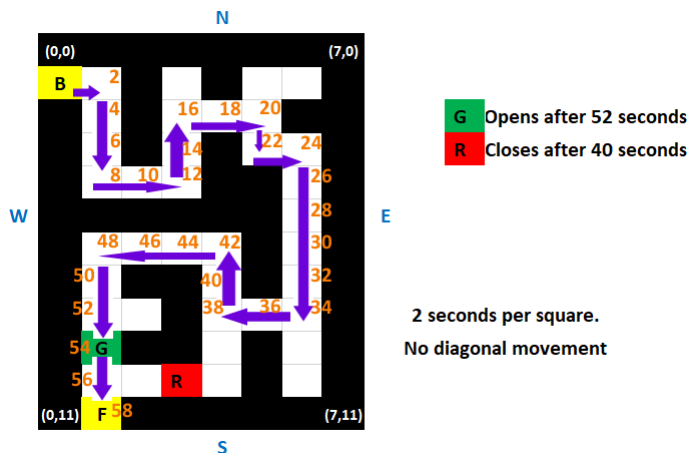
- C is the door color ((**R**)eds close, (**G**reens open)
- N is the number of seconds (from zero) until the door will perform its action
- X is the X coordinate of the door (with 0,0 in the upper left corner of the map)
- Y is the Y coordinate of the door

Walls are shown with the hash (#) symbol. Free spaces are shown with the period (.) symbol. Your starting coordinate point in the map is marked with a **B** (**B**egin). Your ending coordinate in the map is marked with an **F** (**F**inish).

```
8 12 2
#####
B.#.#.#
#.#...##
#.#.#.#
#...##.#
#####.#
#...#.#
#...#.#
#...#.#
#G##.#.#
#..R.#.#
#F#####
G 52 1 9
R 40 3 10
```



Output



From your **(B)**eginning coordinate you, Frodo, must help call out the directions to the vanguard of the fellowship by giving the direction and the number of steps to take next for the **SHORTEST** (by steps) path out of the mines. Gimli has helpfully jotted down the time left from when he hands you the map, until certain doors ahead will open or close.

It takes the fellowship 2 seconds to "step" from one grid coordinate to the next. Due to the way the Dwarves have crafted their tunnels, you will only be able to travel **horizontally** and **vertically** (Dwarves dislike diagonals and dragons). The map Gimli hands you for the section of the mines you are in will never be larger than 15x15 grid squares in size (Gimli has a *lot* of maps). You are

guaranteed to be able to find a way out if you follow your map, and that there will only be one way to do so. So, pay attention to the map, Frodo!

Your output should be a list of directions in the format of ((**N**orth, (**S**outh, (**E**ast, (**W**est) a space, then the number of steps to take. Stop outputting steps when the **(F)**inish coordinate is reached. North will *always* be at the "top" of your map (see picture for example), West to the "left", South at the "bottom", etc.

```
E 1
S 3
E 2
N 2
E 2
S 1
E 1
S 5
W 2
N 2
W 3
S 5
```

Discussion

NO WAITING The Dwarves have cleverly crafted their network of tunnel doors to ensure that, so long as they take the right path, they will never need to wait at a door for it to open. It will be open (or shut) by the time they reach it.

While you would be *technically correct* (the best kind of correct) that the fellowship could wander all over the map and still eventually make it from the beginning coordinate to the finish -- after blindly groping around in the dark to touch every locked door -- you have to remember that an angry (fire-breathing) Balrog is chasing you! **One** misstep could cost the fellowship their lives.

Frodo's job is to read the map and figure out, *before they get there*, which way to guide the fellowship to **avoid doors** which will be shut by the time they would arrive (and, conversely, aim for doors which will be open when they reach them). The output shown in the example here therefore **does not** have the fellowship march up to the closed door, then turn around and try an alternative route. Frodo, wisely, avoids getting the fellowship trapped down a corridor with no escape, and *your code's output needs to do that as well!* (Internally, what your code does to figure out which route to take is between you and Frodo, so long as it doesn't show in your output.)

Reminder: have you run your solution against **all** of the student data sets?

Input 1	Output 1	Input 2	Output 2
7 14 7	N 13	15 15 10	W 1
###F###		#####	S 2
#.R.R.#		#####...#.B	W 2
##.##		#####R#.#	S 1
##G#.#		#####...#	W 3
##.##		#####...G.###	S 3
##G#.#		#RR...###R#.#	W 1
##.##		#.R...###.##	S 1
##G#.#		#G#####.##.##	W 2
##.##		##.#####.##	S 5
##G#.#		##.R#G#R#.#.##	W 5
##.##		##...#.#.#.#.#	
##G#.#		#####.##.##	
#####		#####.##.##	
###B###		F....#...#...#	
R 26 2 1		#####	
R 24 4 1		G 10 10 4	
G 10 3 3		G 30 5 9	
G 8 3 5		G 50 1 7	
G 6 3 7		R 10 11 2	
G 4 3 9		R 12 11 5	
G 2 3 11		R 28 7 9	
		R 34 3 9	
		R 52 2 6	
		R 52 2 5	
		R 52 1 5	

Elizabeth Swann has hidden the coin the pirates sailing on the Black Pearl need to lift their curse. She's jotted down notes for herself to find it again in the form of a logic puzzle. Elizabeth used the following symbols in her puzzle:

- ! means not, and means 1 step north (N)
- & means and, and means 2 steps west (W)
- | means or, and means 3 steps south (S)
- T means true, and means 4 steps east (E)
- F means false, and means JUMP over a hidden trap
- () surrounding any symbols mean to process those symbols first, but have no step meaning

Help Captain Barbosa decipher her notes to find the coin. He knows only the notes which carry the meaning of *true* will lead the way to the coin.



Input

Elizabeth's notes can be up to 80 characters wide on a single line. Evaluate any symbols inside parenthesis first, "nots" second, "ands" third, and "ors" last, working left to right until the whole line has been evaluated.

```
T&T&(!F) | (T&F&!F)
```

Output

If Elizabeth's notes evaluate to **True**, print out the steps to the coin. Else, if **False**, print: "I am disinclined to acquiesce to your request"

```
4 E
2 W
4 E
2 W
1 N
JUMP
3 S
4 E
2 W
JUMP
2 W
1 N
JUMP
```


Discussion

IMPORTANT Order of operations *matters* when you solve this! You need to evaluate the boolean logic in this problem using the following order of operations:

1. Parenthesis
2. Not
3. And
4. Or

However, we're nice, so we guarantee that we won't ever give you a "not" against parenthesis (e.g. $!(T|F) == F$). We also guarantee that you won't have to deal with a double (or more) "not" such as (e.g. $!!T = T$). Finally, we guarantee that you won't have to deal with nested parenthesis (e.g. $(T|(F\&T)) == T$). So, the order you evaluate "nots" vs. "parenthesis" is up to you.

Given all of that, let's walk through one together. Starting with:

```
T&!F&(F|T&F|!F&T&!F|F)|T&(T|T|F|T)&T&T|!T|(F&F&!F|T|!T&F)|T|F&T&F|T&F&T&(F&F&F)
```

You should get this after evaluating the "nots" and the "parenthesis":

```
T&T&T|T&T&T&T|F|T|T|F&T&F|T&F&T&F
```

After you evaluate the "ands" you should be left with a bunch of "or" statements:

```
T|T|F|T|T|F|F
```

If **any** item in the "or" list is true, you can stop evaluating and return true. (This is called "short-circuit evaluation.")

Reminder: have you run your solution against **all** of the student data sets?

Additional Examples

Input	Output
<code>T&T&(!F)&(T&F&!F)</code>	I am disinclined to acquiesce to your request

Elmo wants to paint endless rectangles to practice his counting.

He wants to make the *walls* of the rectangle with numbers, adding one for each additional rectangle. Elmo only knows how to write clockwise. It doesn't matter to Elmo if the completion of a rectangle ends on a full word or not (Elmo's only in preschool after all).



Input

You will receive two (2) integers on single line, separated by a space. Both integers will range from 1 to 100. The first integer will tell you where to start your rectangle output. The second will tell you where to stop.

```
1 6
```

Output

Start by writing out 1 as "one" (always lowercase). Then wrap 2 (as "two") around it, going clockwise, starting above the upper left corner of the previously completed rectangle. Keep going until you have drawn a rectangle for each number from your start and stop input values. Your output should sit flush against the left side of the screen (e.g. don't pad or center the output)

```
isixsixsixsix
sefivefivefis
xvrfourfourvi
iiuethreetfex
sfoetwothofs
xefrwonewruii
ivrhtowtoervx
siuteerhtefes
xfofruofruofi
ievifevifevix
sxixsixsixsix
```

You are guaranteed that your output will fit inside an 80x80 "grid".

Discussion

Numbers should be written out without spaces or hyphens. So, 49 would be fortynine, 99 would be ninety-nine, etc. *There is a number dictionary named numbers.txt included with your student datasets for this problem. Refer to the number dictionary for the text for the integers. The numbers.txt is being provided to save you typing time. You do not have to use it.* You may copy this data into your source code if it helps you. **THIS FILE WILL NOT BE PRESENT WHEN JUDGES TEST YOUR PROGRAM**, so will not be able to read it in as an input file in your program.

Start each layer above the upper-left corner of the end of the last layer (starting point characters shown in red in the example graphic) -- except for the first, which has no prior layer. Work clockwise to wrap the characters of the current layer around the prior layers. Make sure the next layer of rectangle you add *fully encloses* the prior layer:

Correct:

```
otwot
woneu
towto
```

Incorrect:

```
twot
woneu
towto
(There is a gap in the upper left corner)
```

Reminder: have you run your solution against **all** of the student data sets?

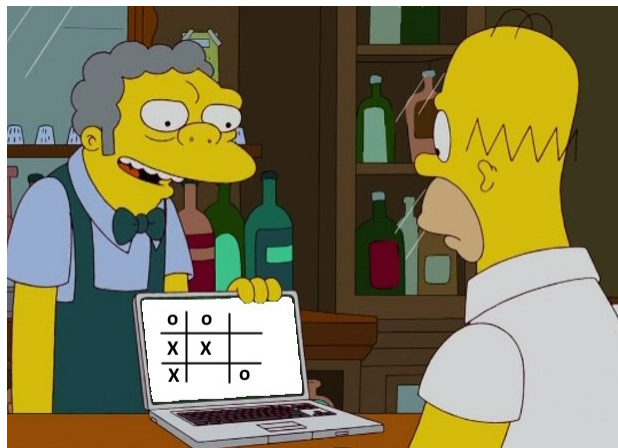
Additional Examples

Input 1	Output 1	Input 2	Output 2
4 10	ntentententent einineninenin tneighteighteie neinsevenseneinn eneessixsixsnget titvxifivefishne nnheiffourixetin eegssevifevsvent tninxisxisxieiee nieevesnevesngnn enthgiethgiethit tenineninenine netnetnetnetnetn	25 32	hthirtytwothirtytwothirt tithirtyonethirtyonethiy ohrthirtythirtythirtytrt wtietwentytynetytynhtw tehngtwentyeighttweeniiyo yntiittwentyseventwtnrot toynenitwentytynetytynht rytyyestwentytynetytynht itrtrtytnewtxisytntiwttr hrinnesytnewtnevesygehhht tiheewtthgieytnewtthniy ohtwteninytnewteninytrrt wtytrihtytrihtytrihtytrt tenoytrihtenoytrihtenoy ytrihtowtytrihtowtytriht



Moe has run into a little trouble running his bar and had to take out a loan from "Fat Tony" D'Amico. Now he's trying to hustle his bar customers in Tic-Tac-Toe to earn enough to pay Tony back at \$5 per game.

Moe's next target is Homer. Lucky for Homer, he has Lisa with him. Unlucky for Homer, Lisa is less interested in helping Homer win bar bets, and more interested in testing the "AI" programming of her new robot.



Input

You will receive a Tic-Tac-Toe game board represented in ASCII characters in 3 lines of characters (X or O or . (a period means an empty space)) On the 4th line will either be the words: **X PLAYS NEXT** or the words: **O PLAYS NEXT**

```
OO.  
XX.  
X.O  
O PLAYS NEXT
```

Output

Lisa's "AI" needs to determine who will win the game. The possible outcomes are:

- GAME WILL BE A TIE
- NOT ENOUGH INFORMATION
- X WILL WIN IN # MOVES
- O WILL WIN IN # MOVES

(Where # is the total number of moves **BOTH** players will need to take before there is a winner) If X or O will win, print out **all** their possible winning 3 spaces in ascending (smallest to largest) order (one set of winning coordinates per line) on the following lines with a single space between each space number (see discussion for space numbers).

```
O WILL WIN IN 1 MOVES  
1 2 3
```

Discussion

If you are unfamiliar with the rules of Tic-Tac-Toe, a player wins if they can line up their symbol (X or O) in 3 touching spaces (horizontal, vertical, or diagonal). The players take turns until spaces to place a symbol run out, or someone wins. There are 9 spaces total on the game board:

```
1 | 2 | 3
--|---
4 | 5 | 6
--|---
7 | 8 | 9
```

Lisa's "AI" can't calculate more than 2 moves out from current conditions, so your programming won't have to worry about that either. If the game's outcome can't be determined within the next 2 moves, give the "NOT ENOUGH INFORMATION" output.

"AI" Order of Operations:

- If a player can win during their move, ignore all other options and win (e.g. do not choose to block instead of winning)
- If a player needs to block or the other player *will win* on their next turn, then block (see note below)
- Else, have Lisa's "AI" choose the next available space working left to right, top to bottom (1->9) to place the next X or O

Note: if a player is going to win in *multiple ways*, and the current player can only block one, then simply ignore the block condition, as the blocking player *will not win*. Skip down to the win condition of listing the ways the winning player can win. Remember, you list the winning state of conditions in your output, not the possible ways a player can lose.

Reminder: have you run your solution against **all** of the student data sets?

Additional Examples

Input 1	Output 1	Input 2	Output 2
OOX XOO O.X X PLAYS NEXT	GAME WILL BE A TIE	OOX O.. X.X O PLAYS NEXT	X WILL WIN IN 2 MOVES 3 5 7 3 6 9 7 8 9

Input 3	Output 3
O..X X PLAYS NEXT	NOT ENOUGH INFORMATION

The CodeWars Problem-Writers have been extremely busy, but they've left quite a mess. To be fair, they did invent a robot to collect all their garbage, but the robot has only 3 actions for a move:

- Turn Left, Turn Right, or Don't Turn (**backward is not an option**),
- Then Go Forward and **stop** at the first square with garbage,
- Then Pick Up The Garbage in that square.

The robot will only stop when it finds garbage. If it hits a wall, it will break.

Of course CodeWars Problem-Writers always create a situation where there is exactly one way to solve a problem. But the robot needs some guidance to find its route.

Given a map of a square room showing the squares with garbage, determine the order in which each garbage-square must be visited to collect all of them. After garbage has been picked up, that square is empty and may be crossed again without stopping.

IMPORTANT: The Problem-Writers have set the garbage in every room so that the start and end squares have only one way to reach another garbage-square. After identifying these two squares, you must decide which is the start square (square 1), and the robot will be placed there to pick up that garbage and face the second garbage-square.

			3K	Q2
		æ		
				Uj
		cø		
m		b	t	ل

			5	6
		1		
				7
		2		
9		3	4	8

Input

You will receive a complete map of a room with garbage. The first line will be an integer N (from 5 to 10), the size of the square room. The next N lines each list N squares. The squares are marked with either "." (empty) or "<>" (garbage). Squares are separated by a single space. The count of garbage-squares will be less than 30.

```
5
. . . . <> <>
. . . <> . . .
. . . . . <>
. . . <> . . .
<> . . <> <> <>
```

Output

Re-print the map, replacing each garbage-square with a number showing the order of collecting the garbage. For a 1-digit number, use a space followed by the digit.

```
. . . . 5 6
. . . 1 . . .
. . . . . 7
. . . 2 . . .
9 . . 3 4 8
```


Additional Examples

Input 1	Output 1
9	
<> <> <> <>	22 21 20 19
.. <> <>	.. 8 9
.. <> <> <>	 14 15 16
.. <> .. <>	 13 .. 12
.. .. <> .. <> <> <>	 1 .. 2 11 10
.. <> <> <>	 3 17 18
.. <>	 4
<> <> .. <> <>	23 7 .. 6 5
<> <> ..	24 25 ..

Inspiration for this problem comes from "Zen Soltiare" in the App "100 Logic Games 2."

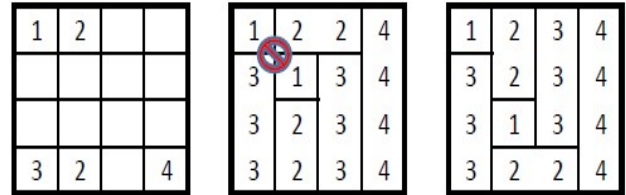
In your new floor-tiling pastime, you've discovered the patterns of Tatamis. A Tatami is a rectangle 1-unit wide and from 1-unit to 4-units long. (Your excitement over a 1-unit wide rectangle "pattern" is a topic we may address later.)

Your business is booming, and your customers have been requesting their own designs. Given the size of a square room, and only a few identifying squares, you have been able to create a unique design for each of them.

There are 2 rules when placing Tatamis:

- A Tatami may not share an edge with a Tatami of the same size.
- Four Tatamis may not share the same corner.

For example, given the partial map on the right, it would seem there are two possible tilings, but the first one fails because four Tatamis share the same corner.



Input

You will receive a partial description of a square room tiled with Tatamis. The first line will be an integer N (from 4 to 10), the size of the square room. The next N lines describe squares already known. Each number marks the only square known in a Tatami of that size. Unknown squares are marked with a period (.). Not all Tatamis are marked.

```
4
12..
....
....
32.4
```

Output

Print the completed tiling of the room.

```
1234
3234
3134
3224
```

Discussion

Remember to test **all** datasets. The provided inputs include difficult 10x10 rooms. To PASS, all datasets must be solved, and the time to solve any single dataset must be less than 1 minute. Judge data is different than student data.

Inspiration for this problem comes from "Mixed Tatamis" in the App "100 Logic Games 2."

Additional Examples

Input 1	Output 1	Input 2	Output 2
6		7	
.2..3.	322333	14.	4444142
...4..	344442	3...2..	3332242
.1....	312232	...4...	4444343
...3.	444431		3331343
..1...	221232	.1.....	2122323
.3..1.	333212	...421	2444421
		..31..3	3331333